

keep focused on our goal, and use blocking issues to indicate the state of each exit criterion.

This template may be used for epics:

text

Participants

Context

Goals

Dependencies and Stakeholders

These teams should be consulted for this project:

These teams should be informed about this project:

Phases of Delivery

Exit Criteria (optional)

Project Management information (optional)

Status yyyy-mm-dd

/labels ~"group::production engineering"

Large Projects

Large projects should be broken down into smaller deliverables.

It is at the discretion of the EM and DRI to choose between the sub-epic and the milestone method for managing large projects.

In Progress

When the EM and PM agree, and there is space for a new project to begin, the chosen project should be removed from the Roadmap issue and associated directly with the team's top level epic.

The start and target date should be set, the DRI assigned, and the participants listed. The workflow label of workflow-infra::In Progress is applied, the start date and target date are set, and the project begins.

Completed

When the project is complete, follow the instructions on the Infrastructure Platforms project management page.

When the epic is closed, it can remain associated to the top-level epic. When we have too many epics associated, or when the end-of-year summary is produced, these epics are moved to the "Completed in Year xxx" epics, of which this is an example.

## Non-Project Work

There are times when the teams are working on smaller issues that do not make up a project.

Always consider if these smaller issues should be batched to demonstrate progress on a particular part of the system. It is OK to have epics that are lower priority and longer running as a way of grouping this work together.

## Issue boards

On the planning board, the goal is to get issues into a state where we have enough information to build the issue.

However, not all issues that are workflow-infra::Ready to be built should be scheduled for development right away. Some issues may be too big, or might not be as important as others. This means not all issues that are workflow-infra::Ready on the planning board will move to the build board immediately.

```
| Planning | Building|
|-----|-----|
| Planning Board | Build Board |
| Issues where we are investigating the work to be done. | Issues that will be built next, or
are actively in development. |
|   <br/> <br/> |   <br/> <br/> <br/> <br/>|
```

---

## SECTION: engineering/infrastructure-platforms/production-engineering/runners-platform/\_index.md

## Mission

The Runners Platform team is responsible for developing and maintaining a fully operational, manageable, and maintainable Runners offering across all GitLab platforms. We focus on the core technology underpinning GitLab's runner infrastructure, including hosted runners for GitLab.com, Hosted runners for Dedicated, while ensuring compatibility with the Cells architecture.

Our mission is to provide an operational foundation with appropriate management interfaces and controls that enable reliable, scalable CI/CD execution while collaborating closely with the Runners application team to deliver features essential for smooth operations and successful product launches.

## Vision

"A single Runner Platform for our SaaS Platforms"

We envision a unified, robust runners platform that delivers consistent, reliable performance across GitLab.com and GitLab Dedicated. Our platform will provide the operational excellence needed to support GitLab's growing CI/CD demands while enabling cost-efficient scaling and comprehensive observability. We aim to eliminate operational silos and create a foundation that allows product teams to focus on user-facing features while we ensure the underlying infrastructure is production-ready, scalable, and maintainable.

## Ownership and Responsibilities

The Runners Platform team focuses on:

1. Creating systems for managing and scaling the GitLab-hosted runners infrastructure
2. Creating systems to optimize runner performance and resource utilization
3. Building platform-level enhancements to improve runner service reliability and availability
4. Collaborating with product teams to ensure runner features meet operational and production readiness requirements
5. Developing tooling and automation for runner management and monitoring

## Collaboration Model

We work in close partnership with the Verify:Runners team, where:

- Runners Platform team provides the management interfaces and infrastructure foundation
- Verify:Runners team focuses on product offerings and determines how to best utilize the platform capabilities
- Both teams collaborate to ensure features are stable, reliable, and production-ready

## Getting Assistance

You can reach us in [grunnersplatform](#)

## How We Work

As a new team within Production Engineering, we are currently establishing our workflows and processes. We'll continue to update this page as our team evolves.

## Common Links

- Slack Channel: [grunnersplatform](#)

---

## SECTION: [engineering/infrastructure-platforms/tenant-scale/\\_index.md](#)

## Vision

The Tenant Scale group is working towards a horizontally scalable, fault-tolerant architecture for gitlab.com.

It is accomplishing this by introducing Cells at the infrastructure layer and Organizations at the application layer, along with Geo for end-to-end resiliency.

## Team Members

### Group Leads

```
{{< member-and-role-by-gitlab "glopezfernandez" "ayufan" "sxuereb" "tkuah" "rymai" "nhxnguyen" >}}
```

### Geo

```
{{< team-by-manager-slug manager="luciezhao" >}}
```

### Organizations

```
{{< team-by-manager-slug manager="mandrewsgl" team="Organizations" >}}
```

### Cells Infrastructure

```
{{< team-by-manager-slug manager="nick-nguyen" team="Cells Infrastructure" >}}
```

### Resources

- Slack (internal only): [stenantscale](#)

### Projects

1. Cells and Organizations project

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/cells-infrastructure/\_index.md

### Vision

The Cells Infrastructure team is responsible for developing key services and components of the Cells architecture. The team also collaborates closely with other infrastructure and development teams on their contributions to Cells.

### Team Members

```
{{< team-by-manager-slug manager="dlogalbo" team="Cells Infrastructure" >}}
```

### How We Work

### Project Management

### Issue Tracking

The Cells Infrastructure team works across multiple GitLab projects such as [gitlab-org/gitlab](#), [gitlab-org/cells/http-router](#), [gitlab-org/cells/topology-service](#), and [gitlab-com/gl-infra/gitlab-dedicated/instrumentor](#). By default, you should open issues that will be owned by

the team under the Cells Infrastructure Team Issue Tracker and apply the group::cells infrastructure label. Move issues for the Cells Infrastructure team from other projects to the team issue tracker when appropriate.

For issues tracked in the team's issue tracker or other gitlab-com/gl-infra projects, we use the workflow-infra:: labels to track an issue's workflow status. Ensure that all issues that are in progress, ready, or need triage have the relevant workflow-infra:: label applied. We use a workflow issue board to track the workflow status of active issues.

Sometimes we'll need to track issues contained in the gitlab-org top-level group, which does not contain workflow-infra:: labels. For these issues, please use the workflow:: labels. We track these issues using a workflow issue board for gitlab-org.

Having two issue boards is not ideal and is a result of our recent reorganization into the Infrastructure Platforms department. Our long-term goal is to minimize the amount of issues that we need to track in the gitlab-org group and to primarily use the team's issue tracker in gitlab-com/gl-infra/tenant-scale/cells-infrastructure/team.

### Workflow Label Mappings

The following are the workflow:: and workflow-infra:: labels that we use and how they map to each other.

| gitlab-org issues                  | gitlab-com/gl-infra issues      |
|------------------------------------|---------------------------------|
| -----                              | -----                           |
| ~"workflow::refinement"            | ~"workflow-infra::Triage"       |
| ~"workflow::ready for development" | ~"workflow-infra::Ready"        |
| ~"workflow::in dev"                | ~"workflow-infra::In Progress"  |
| ~"workflow::in review"             | ~"workflow-infra::Under Review" |
| ~"workflow::blocked"               | ~"workflow-infra::Blocked"      |
| ~"workflow::verification"          | ~"workflow-infra::Verify"       |
| ~"workflow::complete"              | ~"workflow-infra::Done"         |

### Blocked Issues

We use the following guidelines for denoting when an issue is blocked:

- If an issue depends on the completion of another issue, we use the blocked by feature to denote the dependency.
- If an issue was started but requires further input, completion of another issue, etc before progressing, we use workflow::blocked.

### Weekly Review

Each week the EM will schedule time to review the issue boards and team members can optionally attend. The intent of the review is to:

- Ensure that issues have the appropriate workflow labels.
- Check on the status of in progress, blocked, and in review issues.
- Ensure that there are refined issues ready to be worked on. Refined issues are weighted, have

sufficient context in the description, and a workflow label indicating that it is ready to be worked on.

- Check that issues are aligned to our roadmap and status updates on relevant epics.

## Resources

- Slack (internal): gcellsinfrastructure, gcellsinfrastructurestandup
- Cells Infrastructure Team Issue Tracker
- Team Member Assigned Issues Board
- Issue Board gl-infra
- Issue Board gitlab-org

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/cells\_and\_organizations/\_index.md

## Intro

Cells is a new architecture for our software as a service platform. This architecture is horizontally scalable, resilient, and provides a more consistent user experience. It may also provide additional features in the future, such as data residency control (regions) and federated features.

For more information about the goals of Cells, see goals.

## Requirements and Architecture

Cells overall architecture design document.

A key component of Cells is isolated Organizations.

## Roadmap, and DRIs

### Roadmap

<table>

<tr>

<td>

Cells 1.0

</td>

<td>

Cells 1.5

</td>

<td>

Cells 2.0

</td>  
</tr>  
<tr>  
<td>

- For internal customers only
- Organizations are private
- Users cannot interact with other Organizations (including GitLab Org)
- Groups and projects are private in the Organization

</td>  
<td>

- For existing/new customers of GitLab.com
- Organizations are private
- Existing users can interact with private Organizations on Secondary Cells
- Groups and projects are private in the Organization

</td>  
<td>

- Organizations are public or private
- Users can interact with other Organizations
- Groups and projects are private or public in the Organization

</td>  
</tr>  
</table>

## DRIs and Stakeholders

| Stakeholder             | Role                                | Responsibility               |
|-------------------------|-------------------------------------|------------------------------|
| -----                   | ---                                 | -----                        |
| Sabrina Farmer          | CTO                                 | Executive Sponsor            |
| Gerardo Lopez-Fernandez | Engineering Fellow                  | Tenant Scale Engineering DRI |
| Mark Wood               | Acting Group Product Manager        | Tenant Scale Product DRI     |
| Darby Frey              | Staff Fullstack Engineer, Expansion | Sec and Monetization DRI     |
| Kerri Miller            | Staff Backend Engineer, Create      | Core Development DRI         |

## Communication

### Slack Channels

- fcellsandorganizations (internal only): Regular communication

### Meetings

- Group Tenant Scale weekly standup (internal only)

### Status updates

Status updates are automatically rolled up in the relevant epics

- Cells 1.0 epic
- Organizations buildout epic.
- Org Mover epic.

#### Additional Information

#### Cells Fast Boot 2024

We held a Cells Fast Boot in Dublin, Ireland, between 2024-04-23 and 2024-04-24. Below are the artifacts from the event.

#### Agenda, Slides, and Videos

Please use the Unfiltered Google account to watch video recordings.

1. Main agenda (internal only)
1. Introductions, overview, and logistics: Agenda (internal only)
1. Cells Services - Global Service: Agenda (internal only), Slides (internal only), Video (internal only)
1. Cells Services - Routing: Agenda (internal only), Slides (internal only), Video (internal only)
1. Application Readiness - Organizations and Users: Agenda (internal only)
1. Application Readiness - Dependencies and OKR alignments: Agenda (internal only)
1. Deployment: Agenda (internal only), Slides (internal only), Video (internal only)
1. Provisioning: Agenda (internal only)
1. Observability and Runners: Agenda (internal only)
1. Security: Agenda (internal only), Slides (internal only), Video (internal only)
1. Disaster Recovery: Agenda (internal only), Slides (internal only), Video (internal only)
1. Cells Mover and Isolation: Agenda (internal only)
1. Scalability Headroom and Timeline: Agenda (internal only)

#### Decisions

1. No external customers on Cells 1.0, internal dogfooding only. Cells 1.x is the target to onboard new or existing external customers.

#### Artifacts

1. Day 1 recording: Part 1 (internal only), Part 2 (internal only)
1. Day 2 recording (internal only)
1. Database breakout recording (internal only)
1. Organizations breakout recording (internal only)

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/cells\_and\_organizations/cells\_test\_strategy.md



Cells is a project that spans the entirety of GitLab. More information on what Cells is and how it is being developed is on the Cells handbook page. Instead of recreating feature testing done by the other teams, we will reuse and leverage what exists currently and supplement to fill in gaps.

This approach has the following requirements:

- It must feed back useful information to the engineering teams in an efficient, non burdensome way
- It must provide good coverage so we have confidence to release
- It must be easy to add/enhance/change tests
- It works with our current process

## Strategy

The testing strategy for Cells follows our practice of testing at the correct level. The testing will be focused on a couple of efforts:

- Ensuring we have good functional coverage at the appropriate level
  - Reusing existing testing as much as possible to reduce burden on the teams.
- Creating system level E2E tests incrementally to cover boundary conditions
  - These will be built incrementally as features are built, they will be automated and added to the E2E test suite
  - These tests will be included in our existing pipelines and run per their cadence (unless a need is raised to run on a separate pipeline)
- Non functional testing will also be included
  - Ensuring we're building with Observability in mind and include the hooks to integrate to our Observability tools
  - Migration testing, ensuring that the migrations are performant and handle .com scale
  - Performance testing
    - We will use the Observability tools to do performance analysis on the live environments
    - GPT based tests to check against our reference architectures
    - Enhancing our existing pipelines to capture performance metrics to Shift Left performance testing and enable devs to have better visibility into performance concerns

Viewed as a diagram, this approach would look like:

```
mermaid
flowchart TB
  %% nodes
  featuretests[Feature Tests from Development Teams]
  singlecelltests[Cell Specific Functional Tests]
  multicelltests[Cross Cell Functionality Tests]

  %% diagram
  subgraph multiplecell [Multiple Cell Tests]
    featuretests
    singlecelltests
  end
  end
```

```
multicelltests
end
```

A majority of the testing we will need to do will exist at the Single Cell level, and should be covered by existing tests. We may need some additional tests to cover Cells specific functionality that was implemented at the Single Cell level (i.e. Organizations), but we will get a vast majority of the testing needed from the tests written and maintained by other development teams. On top of that will be functionality that spans Cells (i.e. routing to the correct cell), but these will be fewer and smaller because of the reliance on the existing suite.

## Functional Testing

### Cells

#### Feature

This testing is done as part of the day to day work of development, the unit/integration tests added as part of developing the features. The SET can help advise on edge cases / scenarios that should be considered for testing. We currently have one E2E test suite defined: smoke.

#### Single Cell

The Single Cell level has pretty significant existing coverage due to overlap with our existing test suites (Unit, Integration, E2E). Since each Cell is an instance of GitLab, the existing tests will give us good confidence. We will want to ensure that we have tests that cover the impacted features as well as data that is shared across Cells (like Users). The SET can help on developing edge cases / scenarios to cover, ensuring the tests get written at the correct level, writing E2E tests, ensuring the tests get run in CI with the correct feature flags set.

#### Multiple Cells

The Multiple Cell level focuses on scenarios that span Cells that do not elevate up to the Router level. This covers features like:

- A user logged into one Cell URL is logged into a second Cell URL
- Organizations/Groups/Repos that exist on one Cell do not exist on other Cells
- ...

The SET can help with developing these scenarios, writing E2E tests, enhancing our test frameworks to be able to test.

#### Organizations

Organizations provides the capability for a company to have the ability to better organize their repos (the Organizations Blueprint). We will get a level of coverage from the existing E2E suite (verifying that Organizations do not break existing functionality). Some additional testing can be written to verify Organizations specific functionality (i.e. Owner vs User permissions ), many of these can exist at the Unit/Integration layer with a sanity check at the

E2E layer.

## Router

The HTTP Router definition and the SSH Router definition. It will gain coverage by the Single Cell testing, since it will be run through the Router. The Multiple Cells testing will also cover the Router. However there will need to be additional tests written to verify Router functionality (i.e. that a request gets routed to the correct cell). How much of that testing needs to be done at the Unit/Integration/E2E level will be determined. Router level tests will also be covered in the Multiple Cells tests.

## Topology Service

The Topology Service Blueprint definition. It will gain coverage by the Single Cell testing, since logging in from the outside will exercise the Topology Service. If we need further E2E tests specifically on the Topology Service (via API?) needs to be determined.

## Pre-QA Cell

The Pre-QA Cell is being used for deployment testing purposes and may make use of the E2E test suite as part of deployment testing, but as a verification that the deployment was successful rather than as a feature/functionality test. It is deployed in Ring 0.

## Performance Testing

Our performance testing approach is a multi-layered approach that is focused on Shifting Left and Right performance. Shift Left moves performance testing earlier in the process, Shift Right makes data from live environments (production) more Observable so that it can Shift Left.

## Tracking Issues

- Cells 1.0 work items

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/\_index.md

## The Geo Team

Geo is a Premium feature, built to help speed up the development of distributed teams by providing one or more read-only mirrors of a primary GitLab instance. This mirror (a Geo secondary node) reduces the time to clone or fetch large repositories and projects, or can be part of a Disaster Recovery solution.

## Team members

{{< team-by-manager-slug manager="luciezhaoh" >}}

## Stable counterparts

```
{{< engineering/stable-counterparts role="Geo" manager-role="Fullstack Engineering Manager, Geo" >}}
```

## Goals and Priorities

Our priorities are aligned with the product direction. You can read more about this on the [Geo Product Vision](#) page.

Alongside the items listed in our Product Vision, we need to constantly assess issues that our customers bring to our attention. These could take the form of bug reports or feature requests. Geo users are often our largest customers and some rely on Geo as a critical part of their workflow.

We also work constantly to keep housekeeping chores to a manageable level. Where possible, we address these issues as part of a related project. Where this is not possible, we use time around our projects to make this happen.

## Objectives and Key Results (OKRs)

Every quarter the engineering team sets objectives and key results. OKRs are managed in GitLab as of FY24-Q1. The following are links to issues lists for Geo's OKRs.

[OKRs FY25-Q2](#)

[OKRs FY25-Q3](#)

## Geo's Relationship to Disaster Recovery

Disaster Recovery (DR) is a set of policies, tools and procedures put in place to be able to recover from a disaster.

Geo provides data redundancy. The customer will have a redundant copy of data in a separate location. If anything were to happen to their primary instance, a secondary instance still retains a copy of the data.

However, data redundancy is only one part of a complete DR strategy.

High Availability (HA) is also a step towards Disaster Recovery. At the moment Geo does not provide true HA because if the primary instance is not available, certain actions are not possible.

## How to ask for support from Geo

The first line of support will always be the support engineer assigned to the issue raised by the customer. However, at times more expertise is required to resolve the customer concern and a Geo engineer needs to be involved.

This section outlines the process and expectations when requesting support from the team for Geo-related customer support issues.

## Before requesting support

Before submitting a request for support, please review Geo's documentation, the Disaster Recovery docs, the Backup and Restore docs, the Geo Handbook pages, or search through previous customer issues in the Geo Customers Project. The answer to your questions might be found there. Please reach out in the Geo Support Pod Channel `spdpodgeo` first before submitting a RFH.

## Asking a general question

If you have a general question for which you can't find your answer, then feel free to ask your question in the `ggeo` channel on Slack. Please keep in mind that engineers will do their best to support you and answer your question from the top of their heads. If they need to do more research and/or address more complicated scenarios, you will need to create a support issue (see next section).

## Create a support request issue

We like to use issues when customers need help from the Geo Team. This helps us to prioritize work and make sure that we don't lose history and maintain context when the Slack retention policy activates.

We ask requestors to create an issue in the Geo Customers Project.

Please make sure to use and fill the Geo support request issue template for Geo related questions and Backup and Restore support request issue template. As the requestor you only need to fill the Customer Info and Support Question sections.

Other than the Collaboration template, RFHs must be opened by Support Engineers once a support ticket has been opened.

Collaboration template is used for requests to join calls, whether that's from the EM/PM/engineers perspective.

At minimum Zendesk links and logs are especially important. The issue will not enter our normal triage process if they are missing. If no update has been made on an issue for 2 weeks, they will be auto closed by the EM/PM/Assignee.

We also have a process for triaging RFH issues, please see process

If you like, you may assign a priority label to your request. A geo team member or the PM will review this priority assignment during the triage of the issue. Please use the table below as a reference of priority levels and expected response times.

| Priority | Typically used for                                                                                                                                                               | Expected first response time |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| ---      | ---                                                                                                                                                                              | -----                        |
| P4       | General questions that can't be answered on slack by Geo engineers of the top of their heads and will require a bit more investigation.                                          | 2-3 days                     |
| P3       | Customer problems that are not time sensitive (i.e. they have an easy work around) or scheduling time to engage with the customer in the future for them to achieve their goals. | 1 day                        |
| P2       | Customer problems that are somewhat time sensitive and are blocking decisions                                                                                                    |                              |

or progress to be made on the customer side. | 1/2 day |  
| P1 | Fires and emergencies that the customer is experiencing | 1-2 hours |

\ Response times are based on weekdays (excluding holidays) within regular business hours in the time zones that team members are located.

\ Outages and other urgent matters should be channeled through GitLab's incident management processes.

## Common Links

### Documentation

- Geo
- Disaster Recovery
- Planned Failover
- Background Verification
- Geo Glossary

### Issue Lists

#### Missing type labels

The following links lead to Geo team lists of issues to help with identifying missing subtype labels. After opening each link, select the desired milestone filter.

- Issues missing the type label
- Features issues missing subtype label
- Maintenance issues missing subtype label
- Bug issues missing subtype label

### Other Resources

- Issues relating to Geo are mostly to be found on the gitlab-ee issue tracker
- Chat channel; please use the ggeo chat channel for questions that don't seem appropriate to use the issue tracker for.
- Geo YouTube Playlist
- Product Support Requests
- Geo on staging.gitlab.com

### Planning and Process

Our planning and build process is recorded on the process page.

### Demos

The demos are recorded and should be stored in Google Drive under "GitLab Recorded Videos --> Geo Demos".

If you recorded the demo, please make sure the recording ends up in that folder.

## Geo Terminology

See the Geo Glossary.

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/agile-work-items.md

This section describes the approach that the Geo team takes to break down the requirements into work-in-progress items that engineers move through the workflow of implementation.

First, we describe general agile work item terminology to create alignment and use common industry vocabulary. We then map these general terms to the specific work item terminology used by GitLab.

Finally we provide some guidelines and rules of thumb when creating new items and about choosing the right level of granularity for them.

## Agile Terminology

Depending on your previous experience, you may be accustomed to a number of different terms to reference work items in an agile development environment. However, it will usually be the case that a certain hierarchy structure is in place to organize project work in groups of increasingly larger scope.

A common breakdown of this hierarchy in general terms is as follows:

## User Story

We start with the definition of a user story. This is a description of a low granularity piece of functionality or business value that will be delivered to users of the system. It is often expressed in simple terms free of technical jargon or domain-specific terminology, as much as possible.

The point of a user story is that it is as small in scope as possible, while still being a complete piece of functionality. This means that it would be difficult to divide into smaller pieces of work. And if it were to be divided further, at that point it would be difficult to test or feel incomplete from the user's perspective (i.e. it would provide no actual value on its own).

User Stories should not be confused with the concept of Minimal Valuable Change (MVC) at GitLab. An MVC is used to define how much scope of functionality will be delivered by a single iteration of the product, while a user story is a technique or template used to describe individual pieces of business logic in a way that a developer can know when they are done with that part. In this way, an MVC could be composed of multiple user stories.

User stories are typically expressed by a combination of the story itself and a list of acceptance criteria. The story has a structure like:

> As a \<role>, I would like to \<action> so that \<outcome and/or motivation>.

The acceptance criteria is a simple bullet list of points that describe how the system should behave and validate inputs in a way that allows the user to complete the task that the user story describes.

Example:

Say you are building a contact form for a website. The story and acceptance criteria could look something like this:

User Story: As a website visitor, I would like to submit my contact information to the website administrators so that they can answer my questions via email.

Acceptance Criteria:

- The form is accessible through a "Contact Us" link in the top menu
- The form loads into its own page
- The form has 3 fields: Name, Email, Description
- The description field allows basic text formatting (bold, italic, bullet points)
- The form validates all fields as required
- When the user submits the form, an email is sent to admin@website.com with the information submitted.
- The user receives a confirmation message on the screen that the form was submitted

From this example, we can see how we could attempt to separate each piece of criteria into a separate user story, but it would then not provide full value to a user. For example, accessing a form from a menu item which can't be submitted would not be very helpful to the user.

## Feature

A feature in this context is a grouping element of user stories that share a common goal or a similar theme. This collection of stories would address a higher-level job to be done by users of the system. The feature would have a description in this higher level scope and explain how the individual user stories would work together to meet the feature requirements.

Following our example. A feature could be "Support Requests", where the contact form is just one user story of various ways that a visitor to the website could request support. Other user stories in this feature could include searching an FAQ or engaging with a Chat Bot or initiating a live chat session with a customer representative.

## Epic

An Epic is another grouping entity for work. This is a much higher level one and it can include multiple features that deliver on a whole subsystem. Alternatively, it could include features from multiple subsystems that happen to deliver on a cross-cutting job to be done.

An example of an epic might be the development of a customer support portal, which is an entire website with capabilities to browse the documentation, a forum to engage with other customers,



and the Support Request feature cited above.

## Task

Going in the opposite direction, a task provides more granularity than user stories. Tasks are often actually technical in nature and they describe the implementation steps that need to be taken to satisfy the acceptance criteria for a User Story. Typically an engineer would create a task breakdown outlining the changes that the system needs to undergo to consider the work completed.

In our Contact Form example, tasks could include writing the HTML to layout the fields, adding CSS to style the form, writing javascript to do validation and submit the fields, writing a back-end script to process the fields and send the email etc.

## Bug

This hardly requires an explanation. A bug is a defect in the intended functionality of the product or a problem that degrades the performance, user experience or usability of the system. Bugs are usually lower in the hierarchy usually assigned to user stories. However, they can be independent pieces of work in a project, especially when they are detected in production after the initial user story or feature was already deployed.

A bug item should have:

- The pre-conditions or context where the bug applies
- The specific steps for reproducing the problem
- The current outcome and why it is a problem
- The expected outcome, which would make the bug resolved.

## Mapping to GitLab work items

GitLab (the platform) has an approach to project management that is agnostic to development methodologies, which provides for a lot of flexibility. However, this means that there are fewer opportunities to represent in a more explicit way the hierarchy of work items presented in the previous section.

This table aims to provide a mapping between the concepts in the hierarchy and the actual capabilities and features of GitLab.

| Concept    | GitLab Work Item           |
|------------|----------------------------|
| ---        | ---                        |
| Epic       | Parent Epic                |
| Feature    | Epic                       |
| User Story | Issue                      |
| Task       | Tasks                      |
| Bug        | Issue with label type::bug |

\ For tasks, there is a couple of options, we can use task lists in the issue description using markdown, or we can create formally Tasks that are its own work item type in GitLab.

## Guidelines and rules of thumb

### Creating new work items

- For the most part new features will be created by the product manager
- Anyone can create work items. However, please observe the general definitions of such items and use them accordingly.
- If you are an engineer, new work items should be created in the Planning Breakdown workflow and you should then proceed to flesh out the details and provide an estimation of effort. At that point have a discussion with the PM and/or EM about priorities before moving it to the Ready for Dev stage.
- Never move new items to Ready for Dev until they have been fleshed out with details and estimated.

### Using the right scope and granularity

- Use the right concept and type of work item for what you are trying to accomplish.
- It is generally a good rule to start with a user story and then move up the scope hierarchy (i.e. to Feature or Epic) if you think it is getting too big for a single item.
- If you have identified a technical task that needs to be done. Before creating an issue for it, think about the use case that the technical task is serving (i.e. if not already specified by a user story from the PM). Based on this, create a user story that will be solved by completing the task. In other words, put on your PM hat and always describe things from a user-centric perspective. Then add the technical task to the user story.

### Planning for execution

- For Epics and Features, always provide a general description and make sure you have created the full hierarchy of items.
- Planning for Epics and Features takes quite a bit of effort, so before starting your breakdown and fleshing out issues, make sure the work you are proposing has been discussed with the PM and aligns with a short-term product outlook. If not, you are better off leaving it with the PM as a backlog item or a feature request, so it can be fleshed out closer to the time when it will be executed.
- Once an Epic has started, it is better to refrain from continuing to add to the scope. - Always check-in with the PM before adding more scope to Features and Epics.
- Since Epics should have items that have been estimated before execution, use a Due date for the Epic to set an expectation of completion. Update this Due date if necessary as the work progresses.
- Epics should be closed when all work items have been completed. If you discover new scope of work that will need to be done, please open a new work item accordingly so the additional scope is documented and can be planned for a future iteration (i.e. avoid scope creep unless thoroughly discussed and justified).

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/process.md

### Discussions

Discussions are documented in this GitLab project.

## Planning

### Hierarchy of Work Items

One aspect of planning that provides clarity for development execution is the breakdown of roadmap into a hierarchy of informational pieces that are organized in GitLab's own work items as part of product management.

This section describes the approach that the Geo team takes to break down the requirements into work-in-progress items that engineers move through the workflow of implementation.

First, we describe general agile work item terminology to create alignment and use common industry vocabulary. We then map these general terms to the specific work item terminology used by GitLab.

Finally we provide some guidelines and rules of thumb when creating new items and about choosing the right level of granularity for them.

The details of this are documented in our Geo's Hierarchy of Agile Work Items page.

## Kanban

We work in a continuous Kanban manner while still aligning with Milestones.

Epics and issues have the following lifecycle:

```
mermaid
graph LR;
  Plan --> Schedule;
  Schedule --> Build;
  Build --> Release;
  Release --> Post-Release;
```

Three boards are used to monitor this process

- Plan
- Schedule
- Build

### Plan

We use the Kanban Plan board to triage issues that are not yet planned for a milestone. Issues in this board are labeled "group::geo" and use the workflow labels "workflow::validation backlog", "workflow::problem validation", "workflow::design", "workflow::solution validation", "workflow::planning breakdown" and "workflow::scheduling".

The Product Manager(PM) owns this board and uses it to advance issues through to becoming feasible Epics. When work leaves this board, it is in a fit state for the engineering team to implement the issue.

In this stage, the PM establishes new work that they would like to see in the Geo feature. The planning process is as follows:

#### Problem validation

The PM tries to establish that the issue describes a problem with the Geo feature that is customer-relevant (internal and external), or has some other clear merit (e.g. a technical improvement in the backend). The PM will draw on other group members, customers and the UX organisation where necessary. If the problem is validated, the issue is moved forward to the next stage, otherwise it is closed with a short explanation on why the problem will not be considered. This explanation should also contain a link to the category strategy.

#### Solution validation

Once the problem is validated, the PM will consider creating a work item as per our guidelines for hierarchy of work.

They will then reach out to the Engineering Manager(EM) for a technical contact in the team. The EM assigns ownership of the work item to an engineer. The engineer will work with the PM to determine a technical solution for the problem.

Occasionally, a proof-of-concept (POC) is necessary to determine a feasible technical path. When one is required, the PM will create a POC issue that contains the context of the research to be conducted along with the goals of the POC. This issue will be scheduled for work before any further breakdown of tasks is performed.

POC issues are also time-boxed with a due date applied to these items and should be labeled ~"POC". On the due date, the engineer is expected to provide a comment on the issue regarding the outcome of the POC. Time-boxing these issues is meant to constrain the scope of the work.

It is also important to note that not all POCs will be successful, and that is OK! Some avenues of research may not be successful, and the POC will have saved us from investing significant time in a solution that will not meet our needs. The goal is to fail fast!

The PM and the engineer will work together to create all of the work items required to build the solution. These items account for

- Documentation
- Testing
- Post-release items such as rake tasks, migrations or monitoring issues

Ideally, epics, features and user stories should be broken down by external functionality rather than by implementation details. Even refactors and performance improvements should highlight the customer value and be described with a user-centric approach.

Post-release tasks that need to be performed by SRE's are created in the Infrastructure team project and cannot be added to work items as related issues. For these, it's useful to list them in the highest level work item description.

When they are satisfied that they have a reasonable solution including an estimation of effort for each work item, this breakdown should be shared with the rest of the team for discussion and feedback. Planning is transparent anyway, but this serves to notify the team that the new work items are ready for development.

For issues that include a feature that uses a Feature Flag (FF), we follow the feature flag lifecycle doc.

The original issue should remain open and blocked on the FF rollout issue until the rollout issue is closed, in case the rollout does not go as planned.

## Schedule

On an ongoing basis, the PM and EM will meet to discuss the work that is currently active, and queue prepared items for development.

In this process, prepared items will be given the milestone label along with workflow::ready for development, which will pull items into the Build board.

Part of the ongoing prioritization and scheduling effort includes providing answers to these questions

1. What is active now?
1. What needs to be active next?
1. Are there any deliverables?
1. Is the active list in priority order?
1. Bugs list
1. Technical debt items arising

The outcome of these scheduling activities are reflected in an Outlook Issue that the PM prepares and which encapsulates the direction and scope of the upcoming iteration.

## Build

We use the Kanban Build board to look at issues that have the %Started milestone in combination with the group::geo label.

The Engineering Manager(EM) owns this board and uses it to facilitate building out the Epics and issues that the team have decided are ready for development.

Issues are added in the "ready for development" column in priority order. When an engineer is open, they can pick unassigned issues from the top of this list. As their work progresses, they advance the issue through the "in dev" and "in review" columns. Engineers need to keep the issues aligned with the status of the corresponding merge requests.

When an issue is "in review", the MR should be assigned to both the original engineer and the reviewer so that it is clear that the merge request has an active reviewer. This helps the EM to see how much work is allocated to each person.

The column for "verification" is where the owner of the Epic, or the PM will ensure that the issue fits into the context of the solution. This is also where we confirm if a release note is required for this issue, and assign the correct milestone.

### Release and Post-Release

For some issues, there are tasks that need to be performed by SRE's after the release is completed. Because these tasks are often in the infrastructure project, they cannot be added to Epics. We still need to track these and follow them through to completion.

Finally, we ensure that any communication that needs to go out for this issue is delivered. These can take the form of release posts, blog posts, video tutorials, or demos.

### Work Ownership

The Geo team uses a variety of work items to describe features or capabilities that will increase the maturity of the Geo categories over time.

Each high-level work item (i.e. Epics and/or Features) should be owned by an engineer who is responsible for all technical aspects of that work item. If at any point, the owner needs to take leave that is more than a few days, they should assign another engineer to act as owner until they return.

In the planning phase, the engineering owner will work closely with the Product Manager to understand what the requirements are and why they are important to customers. The engineer will decide on how best to deliver the solution and create Tasks that encapsulate the technical work required. They may need to consult with other team members and stable counterparts to come up with the right approach to delivering the requirements.

They should include Tasks for documentation changes and additional testing requirements that are required. A task should be created to inform Geo experts in the Support group about the

change if needed. The engineer should also consider if there is any technical debt that is appropriate to address at the same time. Additional Tasks are also needed for any rollout or post-release todos.

The engineer that is the owner does not need to be the only person implementing the Tasks. They should keep watch of the work that is done on the tasks so that they can verify that the work is progressing correctly.

Ideally, they should also be an approver on each MR for the work items. If there are problems with the work, or lengthy delays, they need to make sure the Product Manager and Engineering Manager are aware.

When work is nearing completion, the owner engineer should check the release note and work with the PM on any changes.

They should also make sure that any additional issues that may have come up during the build process are either addressed, or scheduled for work. This will help to make sure that we do not build up technical debt while building.

Finally, owners should also monitor any work that needs to occur while rolling out the Epic or Feature in production. If there are

rake tasks, database migrations, or other tasks that need to be run, they need to see those through to being

run on the production systems with the help of the Site Reliability counterpart. They may also need to assist the

Product Manager with creating the release note for the work item.

This places a lot of responsibility with the owner, but the PM and EM are always there to support them. This ownership

removes bottlenecks and situations where only the PM or EM is able to advance a work item. In addition, the best people

to decide on how to implement a task are often the people who will actually perform the work.

The responsibility of the Product Manager is to make sure that there is enough information for the engineer to develop the

correct solution that meets the requirements. They are also available to answer clarifying questions or consider how to

approach edge cases. At the end of a work item they also communicate this out to customers and other interested parties.

The Engineering Manager is responsible for clearing the path. They need to make sure the engineers performing the work

have the access to the right information, people, tools, and other resources to get the work done. They try to foresee

problems and clear any blockers that may arise while the work is in progress.

## Opening an Issue

Within the Geo Team we utilize Issue Templates to create consistency in our Backlog to help us be more Efficient and deliver more Results.

Using Issue Templates has been found to help the team in the following ways:

1. Ensures an Issue has all the required information so that any contributor can pick up and start or join a conversation without any outside context.
2. Empowers our Refinement Process to work more efficiently which results in higher Community Contributions and less SME reliance.
3. Helps Issues stay relevant and not unintentionally sink to the bottom of our Backlog.

We use the following Issue Templates:

- Bug (template)
  - Used as the template for Bugs and their investigation.
- Feature - lean (template)
  - Used as the template for larger Feature requests. These will often drive a conversation and ultimately be promoted to Epics with Implementation Issues being broken off into MVC changes.
- Implementation (template)
  - Used as the template to break down large Epics, organize MVC level changes, and assist in our Refinement Process.

Note: Many sections in a template should be treated as optional if there is no relevant information to be added.

#### Addressing new issues quickly

When new issues arise (through testing, customer support issues, or other means) we still want to be able to address them quickly without being delayed by process.

If you think a new issue needs to be worked on immediately:

1. Make sure there is enough detail on the work item description for someone else to understand the issue and for someone else to have enough context to review your work
2. Make sure there is a weight
3. Assign it into the current milestone
4. Add the workflow::STATUS, group::geo and the unplanned label
5. Assign it to yourself

#### Weights

We use the weights from the set [1, 2, 3, 4, 5, 10] where the value represents a "best work day". A "best work day" means

a day where there are no interruptions, emails or other demands on your time.

For example, if an issue is assigned a weight of 5 then an engineer should be able to complete the work in 5 days if this were the only thing that they needed to do for those five days.

If an issue is assigned a weight of 10, then the issue is too big and will need to be broken down further. This usually means promoting the issue to an epic and raising individual issues that have a lower weight.



If any issue is weighted above a 3, we should ask ourselves if it can be further broken down. We should do this even if the issue was already broken out from an even larger issue.

## Backlog Refinement Process

Backlog refinement equals moving issues from workflow::validation backlog through each stage toward workflow::ready for development. Engineers will follow instructions provided in a weekly assigned "refinement issue", which generally aligns with Product Development Flow.

Issues in the GitLab.org group labeled ~"group::geo, workflow::validation backlog will be refined.

Each week 3 issues will be randomly selected by a bot and refined by the team. Bugs will be prioritized over feature requests and a go/no go will be given.

1. Refinement issue is created and assigned to engineers. It contains instructions for what to do with each issue and is the source of truth for this part of the process. The rest of the steps below are an overview.

2. Phase 1: Engineers will select 1-3 issues each and put the label ~"workflow::problem validation on when they start taking a look.

1. If the issue doesn't have the right issue template/enough details, it will be sent back to the author/PM for clarification

2. If the issue is a go, move to ~"workflow::refinement for Phase 2. See below how to decide if an issue is a "go" or "no-go".

3. Phase 2: Engineers will add an implementation guide, the right labels and weights for the issue. Once that is ready, move it to ~"workflow::ready for development for the PM/EM to schedule.

1. For bug issues, the bug first needs to be reproduced as a part of the refinement process. If the bug issue isn't reproducible, the issue can be closed off. Bug issues also need to be assigned a priority/severity in addition to the regular labels.

1. For features/maintenance issues, a timebox of 1 hour or so is expected per issue and for bugs this could take 2-3 hours as we need to reproduce it before adding implementation details

1. Please also label as ~"Seeking Community contributions" if an implementation guide has been added and weights are 1-3.

1. If this issue has a weight of 1-3 and could be easy enough for a community contributor, it should also be relatively easy to write up the implementation guide. Timebox to 30 mins - 1 hour.

1. If the issue is very complex, stop investigating as soon as you've realized that it's going to take > 1 hour to reproduce/write the implementation guide.

1. If an issue takes too long or you think is not aligned with our current roadmap, be vocal about pushing back on it and discuss it on the refinement issue.

### Phase 1: How to decide if an issue is a "go" or "no-go"

In phase 1, we decide whether or not an issue is a "go" or "no-go" for refinement in phase 2. An issue that is a "go" will always be moved to phase 2 for refinement.

An issue is a "no-go", if,

1. the feature is not aligned with our current roadmap (i.e. in the next 6 - 12 months)
1. the issue's context is outdated or irrelevant now

1. it is a duplicate (mark it as a duplicate)
1. it is already resolved, completely, or enough to not be worth prioritizing any more
1. the solution is complex and the demand is low

Those issues that are a "no-go" should be closed with the reason mentioned or de-prioritized to priority 4, so that phase 2 engineers do not pick them up for refinement. Please tag the EM and PM when closing an issue if you're not 100% confident what to do with it.

The refinement script can be found here

### Engineering Customer/Support Rotation Process

Every two weeks, a Geo engineer is assigned to be the DRI for doing the technical assessment of customer support tickets + monitoring ggeo channel for support issues.

Process summary:

- Every week, a slack reminder in geo-lounge channel will let the group know that a new support shift is starting for technical assessment triage.
- Every Geo engineer is expected to be aware of their upcoming rotation (as per the schedule below) and take action as per the slack reminder.
- The DRI currently assigned to the rotation should then dedicate their two weeks to review issues from the customer support backlog and support in the ggeo channel.
  - Be first on point to triage questions in our ggeo channel. Now that we have a slack workflow -> this should be taught/triggered per more complex questions and quickly answer anything that's short and sweet.
  - Expected duties are: triaging, creating issues, documenting initial research, adding priority labels etc. so that a customer support issue can enter our usual workflow.
- If the DRI is unable to perform an upcoming triage rotation shift due to any reason (e.g. PTO, sick leave, other responsibilities taking precedence), they are expected to swap their rotation with another team member or notify the EM to facilitate. Once the swap is identified, the schedule should be updated with via a MR.
- The DRI needs to update this issue for the 2025 rotation.
- The DRI should estimate the amount of time spent on support and track this via Time Tracking in each issue approximating to the hour how much they have spent on tickets and triaging. This is a new part of the process which is in a trial phase.

At the end of the rotation, each engineer should provide handover notes inside the issue:

- Using our standardized Duo Chat prompt for handing over an RFH (WIP)
- Proofread and correct Duo's output if needed.
- Post it directly on the issue, with a ping to the new DRI, in a new root comment, not on a thread
- If needed, the new DRI should ask clarifying questions on a reply to the comment or in Slack. Setting up a meeting to go over more difficult context would work if timezones aligned.

### Schedules

Schedules are tracked in a Google Sheet(internal link).

## Retrospectives

Retrospectives are a crucial component of the agile methodology. However, having a retro should not be about checking-off a mark in our agile todo list. The purpose of the retro is to learn and then take action that leads to a better place. We learn from our past actions and results and we use that knowledge to improve our future execution.

At GitLab there is a general guideline to conduct group retrospectives and a process automation to run retrospectives asynchronously. The Geo team however is working with a new process that combines elements of fully asynchronous work with the benefits of active conversation and the increased level of engagement that these types of interactions bring.

The details of this process is documented in our [Geo Team Retrospectives](#) page.

## FAQ

What work should I pick up next?

"Pull from the right". That means, start at the right hand side of the build board, and move towards the left.

In order, that means:

- Can you help anyone with reviews?
- Can you unblock anyone in development?
- Pick from the top of the "ready for development" column on the Kanban Build board.

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/retrospectives.md

Retrospectives are a crucial component of the agile methodology. However, having a retro should not be about checking-off a mark in our agile todo list. The purpose of the retro is to learn and then take action that leads to a better place. We learn from our past actions and results and we use that knowledge to improve our future execution.

The Geo team will have monthly retrospectives as a way to reflect on how things are working out for the team and for its members. The objective is to learn and become a better team in terms of collaboration and execution of the team's mission.

> Retrospectives enable Actionable Team Learning. By Learning, I mean that the retrospective must lead to the acquisition of new knowledge or skills. By Team Learning, I mean that the learning must be for the whole team and not isolated to an individual. And by Actionable Team Learning, I mean that the knowledge or skills that the team has acquired must have the potential to lead to change. - David Horowitz (Retrium CEO and Co-Founder)

## Activity and Cadence

The retrospective activity may look different from one month to another by applying different techniques according to what is happening in the team at different times. It is also a way to

keep the retros engaging and avoid burnout from repetition.

Although the retrospective will occur on a regular cadence, the Geo team does not follow sprints but rather a Kanban process. This means that the insight and discussions in the retros do not necessarily need to be limited to a particular time frame.

GitLab promotes the value of remote and asynchronous work. Our retrospective process respects this value. However, the time for reflection is better served by dialogue that happens in a more synchronous way. It is essential that team members can engage with one another in meaningful discussions that lead to actionable insights. This is why although the process is built with support for offline collaboration, there is an established schedule for the different phases of a retrospective and all team members are expected to participate following the schedule.

### Participants

Geo team members required:

- All geo engineers
- Engineering manager
- Product manager

Optional:

- Software engineer in test
- Product designer

Occasional (by invitation)

- Tenant Scale group product manager
- Tenant Scale group director of engineering

### Schedule

Retrospectives will take place from Monday to Thursday during the last week of every month (i.e. the week that includes the last Thursday of the month). Overall this will be about a 2.5 hour time investment spread out throughout the week.

Retrospectives will have 4 phases:

#### Check-in (Monday)

During check-in phase the engineering manager will set the stage by describing the activity for the retro, define the context or scope of the discussion and post a check-in question that all team members must answer. The check-in question can be an ice-breaker or a work-related question that helps people get engaged into the retrospective.

This message will be sent through slack and all team members are expected to take action before the end of day Monday within their respective time zones. Reading the activity and answering the question should not take more than 5 minutes.

## Data Gathering (Monday - Tuesday)

Right after check-in, members are expected to participate in the data gathering phase. The type of data will depend on the activity at hand. Generally speaking it will involve working individually to brainstorm and record data points into a retrospective board as sticky notes. At times, data gathering may involve fetching reports on metrics or other information from outcomes and events that happened in the last period. Data gathering should not take individual members more than 25 minutes and it can happen at any time between Monday and Tuesday.

## Insights (Wednesday)

This phase is where the meaningful analysis of the data happens. This is where team members collaborate to learn from each other and where they find actionable insights.

Throughout the day, team members are expected to read all the data collected in the board as sticky notes and start the conversation by asking each other questions in written conversation threads (i.e. comments) in the context of specific issues. It is helpful during this phase to ask clarifying questions or ask the "why" of the problems identified to try to reach root causes.

By the end of day, all team members should take a last look at the board and make sure they have seen the latest comments that may have come in later in the day. Overall team members are not expected to take more than 1 hour this day to engage in insight generation.

## Actions and wrap-up (Thursday)

On the last day of retrospective the team will have a call to go over the insights discovered during the week and determine if any actions are required and what kind of action is needed. During this meeting the team will also vote and prioritize the actions that are identified so they can be assessed and detect what is important and what is feasible to do in the short term.

All team members are strongly encouraged to attend. This is a once a month meeting and a reasonable commitment for synchronous communication. In order to accommodate for various time zones, there will be two options for wrap-up calls for team members to attend. One will be friendly to anyone located in the US/EAST-WEST time region and another one for the EMEA region. As the team locations change over time, we will adjust the schedule to make sure the meetings always happen within normal working hours.

The retrospective will wrap up with a check-out activity that helps with reflection on how the retrospective itself went and if any adjustments are required.

## Exceptions to the schedule

For each one of these stages, if people are out on PTO, they can be excused from following the schedule and they can catch up with the process when and if they come back from PTO before the retrospective is over. You are encouraged to plan ahead and check-in with the engineering manager the week before if you want to provide your input before leaving for PTO.

## Privacy and Transparency

GitLab promotes the value of transparency, so the activities and data collected for retrospectives will be public by default. However we do want to create a safe space conducive of honest feedback and a high degree of engagement. During Wrap-up, there will be an opportunity to bring up anything that should be shared privately and a new issue can be created with proper permissions to continue the conversation there. Alternatively, you may make use of GitLab's internal notes to have those conversations in the same issue.

Highlights of the retrospective will be written to a GitLab issue accessible to team members as record keeping.

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/scheduled\_pipelines.md

## Summary

Using a combination of GET and gitlab-qa, a series of scheduled pipelines are periodically run to check if there are any breaking changes in Geo. It is necessary to check the Alerts to check for errors, and handle them as appropriate.

## Pipelines

| Title                  | Description                                           | Frequency                         |
|------------------------|-------------------------------------------------------|-----------------------------------|
| -----                  | -----                                                 | -----                             |
| 1k QA                  | Runs Geo QA steps against a 1k Reference Architecture | Inactive                          |
| 3k QA                  | Runs Geo QA steps against a 3k Reference Architecture | Inactive                          |
| 1k Failover & Recovery | Tests failover on a 1k Reference Architecture         | Inactive                          |
| 1k Rebuild UTC         | Destroys and recreates the 1k Reference Architecture  | Daily at 4AM                      |
| 3k Rebuild 4AM UTC     | Destroys and recreates the 3k Reference Architecture  | Mondays at 4AM UTC                |
| 3k Update              | Upgrades GitLab on the 3k Reference Architecture      | Tuesday through Friday at 4AM UTC |

## Handling

1. Check the alert list for triggered, unassigned alerts
1. Acknowledge any unhandled alert, and assign it to yourself
1. Check the failed pipeline to determine a cause of the failure
  1. All of the instances are built in the gitlab-qa-geo-ci GCP group.  
Geo engineers should have access to the group, and can log onto the individual VMs to

access the logs as required.

1. If there is no issue, mark the alert as resolved
1. If there is an issue, create an incident to track the resolution
1. If multiple alerts are being triggered for the same reason, only create one incident
1. If it is a simple fix, fix and close the incident
1. If it is not a simple fix
  1. Open a follow up issues to handle the fix
  1. Quarantine the failing test
  1. Close the incident

---

## SECTION: engineering/infrastructure-platforms/tenant-scale/geo/staging.md

Summary

{{% alert color="warning" %}}

Geo on staging is deprecated, you can find it currently enabled on staging-ref. All the remaining information in this document is outdated and kept for historical purposes.

{{% /alert %}}

Geo is fully operational on the staging environment of GitLab.com. This is a major dogfooding effort that allows the team to test and validate new features at scale, catch bugs, and identify performance issues. This is important for our customers because it increases our confidence in any Geo improvements we make.

The following items describe some spe